

VTU MCA 2022/2024 SCHEME • QUESTION BANK WITH COMPLETE ANSWER KEYS

QB 06: Data Structures & Algorithms — Question Bank with Answer Keys

Course: Data Structures and Algorithms (Theory + Lab)

Target: VTU MCA Semester 2 (Core IPCC)

Scheme: VTU 2022 / 2024 Scheme

Contents: Infix to Postfix Trace, AVL Rotations, Dijkstra's Shortest Path, MergeSort/QuickSort, 15 MCQs with Explanations

Syllabus Outline (VTU 5 Modules)

- **Module 1:** Asymptotic Notations (O , Ω , Θ), Stacks (Operations, Array Implementation, Infix to Postfix Conversion, Postfix Evaluation).
- **Module 2:** Queues: Linear Queue, Circular Queue (Full & Empty conditions), Double-Ended Queue (Deque), Priority Queue.
- **Module 3:** Linked Lists: Singly Linked List, Doubly Linked List, Circular Linked List, Reversing a Linked List, Polynomial Representation.
- **Module 4:** Trees: Binary Trees, Binary Search Trees (BST), Tree Traversals (Inorder, Preorder, Postorder), AVL Trees (Rotations: LL, RR, LR, RL), B-Trees.
- **Module 5:** Graphs & Sorting: BFS, DFS, Dijkstra's Algorithm, Prim's and Kruskal's MST, Sorting (Merge Sort, Quick Sort, Heap Sort) with Time Complexities.

SECTION 1: High-Yield MCQs with Answer Key

Q1. What is the worst-case time complexity of QuickSort when the pivot chosen is always the minimum or maximum element?

- A) $O(N \log N)$
- B) $O(N)$
- C) $O(N^2)$
- D) $O(\log N)$

Answer: C

Explanation: If the partition is extremely unbalanced (e.g., sorted array with first element as pivot), the recursion tree has depth N , yielding $O(N^2)$ worst-case time.

Q2. What data structure is fundamentally used to implement Breadth-First Search (BFS) in a graph?

- A) Stack
- B) Queue
- C) Priority Queue
- D) Binary Tree

Answer: B

Explanation: BFS explores nodes level by level in First-In, First-Out order, requiring a Queue. DFS uses a Stack (or call-stack recursion).

Q3. In an AVL Tree, what are the permissible values for the Balance Factor of any node?

- A) $\{0\}$ only
- B) $\{-1, 0, +1\}$
- C) $\{-2, 0, +2\}$
- D) Any positive integer

Answer: B

Explanation: The balance factor is $\text{Height}(\text{Left Subtree}) - \text{Height}(\text{Right Subtree})$. An AVL tree requires balance factors to strictly stay within $\{-1, 0, +1\}$.

Q4. Which of the following data structures is most suitable for evaluating arithmetic expressions in Postfix notation?

- A) Queue
- B) Stack
- C) Tree
- D) Hash Table

Answer: B

Explanation: Operands are pushed onto a Stack; whenever an operator is encountered, the top two operands are popped, evaluated, and the result is pushed back.

Q5. What is the condition for a Circular Queue of capacity N (using 0-indexed array) to be FULL?

- A) `front == rear`
- B) `(rear + 1) % N == front`
- C) `rear == N - 1`
- D) `front == (rear + 1)`

Answer: B

Explanation: In a circular queue, `(rear + 1) % N == front` indicates that incrementing `rear` would collide with `front`, signifying that the queue is full.

Q6. Which sorting algorithm is guaranteed to have a worst-case time complexity of $O(N \log N)$ and is Stable?

- A) QuickSort
- B) HeapSort
- C) MergeSort
- D) SelectionSort

Answer: C

Explanation: MergeSort has $O(N \log N)$ time in worst, best, and average cases, and maintains relative order of equal keys (Stable). HeapSort is not stable.

Q7. What is the time complexity to search for an element in a balanced Binary Search Tree (AVL tree) with N nodes?

- A) $O(1)$
- B) $O(N)$
- C) $O(\log N)$
- D) $O(N \log N)$

Answer: C

Explanation: In a balanced tree, height is strictly bounded by $1.44 \log_2 N$, so search, insert, and delete take $O(\log N)$.

Q8. What does Inorder traversal of a valid Binary Search Tree (BST) produce?

- A) Elements in descending order
- B) Elements in sorted ascending order
- C) Random order
- D) Level-by-level elements

Answer: B

Explanation: Inorder traversal visits Left Subtree $\rightarrow$ Root $\rightarrow$ Right Subtree. In a BST, this systematically visits keys in ascending sorted order.

Q9. Dijkstra's Shortest Path algorithm fails or may produce incorrect results if the graph contains:

- A) Disconnected components
- B) Negative edge weights
- C) Cycles
- D) Directed edges

Answer: B

Explanation: Dijkstra's greedy assumption presumes edge weights are non-negative. Graphs with negative weights require the Bellman-Ford algorithm.

Q10. What is the auxiliary space complexity of MergeSort on an array of size N ?

- A) $O(1)$
- B) $O(\log N)$
- C) $O(N)$
- D) $O(N^2)$

Answer: C

Explanation: Standard MergeSort requires a temporary buffer array of size N to merge sub-arrays, giving $O(N)$ auxiliary space.

 SECTION 2: Short Answer Concepts (4–6 Marks)
Q11. Explain Infix to Postfix conversion using a Stack. Trace the expression: $(A + B) * C - (D - E) * (F + G)$ **Answer:****Algorithm Rules:**

1. Print operands directly to the output.
2. If $($, push to stack.
3. If $)$, pop and output operators until $($ is encountered. Pop and discard $($.
4. If operator: Pop operators of greater or equal precedence from the stack to the output, then push the current operator.
5. At the end of input, pop all remaining operators from the stack.

Step-by-Step Trace Table:

Symbol	Action	Stack Content	Postfix Output
$($	Push	$($	
A	Output operand	$($	A
$+$	Push	$($ $+$	
B	Output operand	$($ $+$	$A B$
$)$	Pop until $($	$($ empty	$A B +$
$*$	Push	$($ $*$	
C	Output operand	$($ $*$	$A B + C$
$-$	Pop $*$ (higher precedence), push $-$	$($ $-$	
D	Output operand	$($ $-$	$A B + C * D$
$-$	Push $-$	$($ $-$ $-$	
E	Output operand	$($ $-$ $-$	$A B + C * D E$
$)$	Pop until $($	$($ $-$	$A B + C * D E -$
$*$	Push $*$ (higher than $-$)	$($ $-$ $*$	
F	Output operand	$($ $-$ $*$	$A B + C * D E - F$
$+$	Push $+$	$($ $-$ $*$ $+$	
G	Output operand	$($ $-$ $*$ $+$	$A B + C * D E - F G$
$)$	Pop until $($	$($ $-$ $*$	$A B + C * D E - F G +$
Eof	Pop remaining $*$, then $-$	empty	$A B + C * D E - F G + * -$

Final Postfix Expression:

$$\mathbf{\$A\;B\;+\;C\;*\;D\;E\;- \;F\;G\;+\;*\;- \$}$$
Q12. Write a C/Java function to reverse a Singly Linked List iteratively.**Answer:**

```

#include <stdio.h>
#include <stdlib.h>

typedef struct Node {
    int data;
    struct Node *next;
} Node;

// Iterative Reversal in O(N) Time and O(1) Space
Node* reverseList(Node *head) {
    Node *prev = NULL;
    Node *curr = head;
    Node *next = NULL;

    while (curr != NULL) {
        next = curr->next; // 1. Store next node
        curr->next = prev; // 2. Reverse current pointer
        prev = curr;      // 3. Move prev forward
        curr = next;      // 4. Move curr forward
    }
    return prev; // New head of the reversed list
}

```

SECTION 3: VTU Model Exam Problems (10–12 Marks)

Q13. [VTU Model QP - Module 4: AVL Tree Construction & Rotations]

Construct an AVL Tree by inserting the following sequence of keys one by one:

14, 17, 11, 7, 53, 4, 13, 12

Show balance factors and rotations performed at each imbalance step.

Answer:

- **Insert 14:** Tree: (14)[BF=0]
- **Insert 17:** Right child. Tree: 14 → 17 . Balanced.
- **Insert 11:** Left child. Tree:

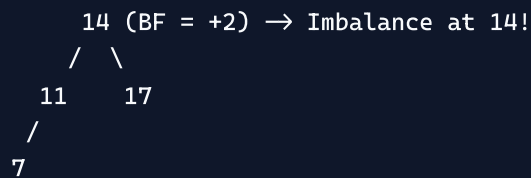
```

      14 (0)
     /  \
    11   17

```

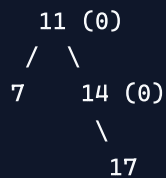
Balanced!

- **Insert 7:** Left child of 11.



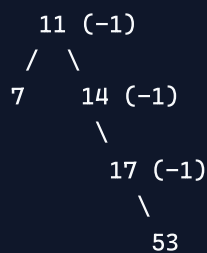
Type of imbalance: **LL (Left of Left)** at node 14.

Fix: Perform **Right Rotation (RR)** at node 14:



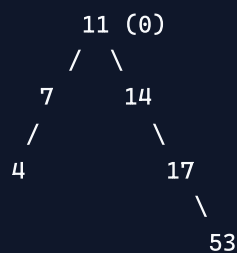
Balanced!

- **Insert 53:** Right child of 17.



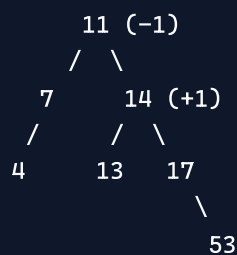
All balance factors within $\{-1, 0, +1\}$. Balanced!

- **Insert 4:** Left child of 7.



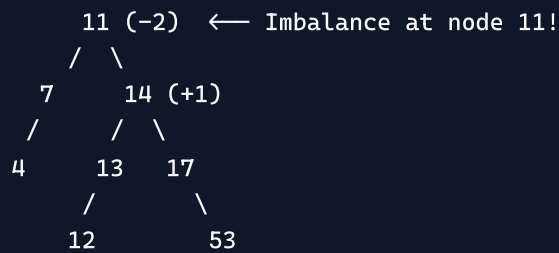
Balanced!

- **Insert 13:** Right child of 11's right child (14). Left of 14.



Balanced!

- **Insert 12:** Left child of 13.



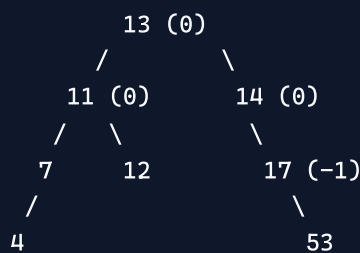
Path to newly inserted node: Right of 11 $\rightarrow$ Left of 14.

Type of imbalance: **RL (Right-Left)** Imbalance!

Fix: Perform Double Rotation:

1. Right rotate at node 14.
2. Left rotate at node 11.

Final Balanced AVL Tree:



All balance factors: 0 or -1. Perfectly balanced!

Q14. [VTU Model QP - Module 5: Dijkstra's Shortest Path Algorithm]

Find the shortest path and distance from source vertex **A** to all other vertices in the weighted directed graph given by the following adjacency list:

- $A \rightarrow B (4), ; A \rightarrow C (2)$
- $B \rightarrow C (1), ; B \rightarrow D (5)$
- $C \rightarrow B (1), ; C \rightarrow D (8), ; C \rightarrow E (10)$
- $D \rightarrow E (2), ; D \rightarrow Z (6)$
- $E \rightarrow Z (3)$

Answer:

Step-by-Step Distance Vector Table:

Initially: $\text{Dist}[A] = 0$, all other vertices $\text{Dist} = \infty$. Visited set $S = \{A\}$.

Iteration	Min Vertex Picked	Distance to A	B	C	D	E	Z	Explored / Relaxed Edges
Initial	-	0	∞	∞	∞	∞	∞	Start at A
1	A							

(Dist 0) | **0** | 4 | 2 | ∞ | ∞ | ∞ | Relax \$A \to B(4)\$, \$A \to C(2)\$ | | 2 | **C** (Dist 2) | 0 | **3** | **2** | 10 | 12 | ∞ | Relax \$C \to B(2+1=3)\$, \$C \to D(2+8=10)\$, \$C \to E(2+10=12)\$ | | 3 | **B** (Dist 3) | 0 | 3 | 2 | **8** | 12 | ∞ | Relax \$B \to D(3+5=8 < 10 \implies 8)\$ | | 4 | **D** (Dist 8) | 0 | 3 | 2 | 8 | **10** | **14** | Relax \$D \to E(8+2=10 < 12 \implies 10)\$, \$D \to Z(8+6=14)\$ | | 5 | **E** (Dist 10) | 0 | 3 | 2 | 8 | 10 | **13** | Relax \$E \to Z(10+3=13 < 14 \implies 13)\$ | | 6 | **Z** (Dist 13) | 0 | 3 | 2 | 8 | 10 | 13 | Destination reached! |

Final Shortest Distances & Paths from Source \$A\$:

- Shortest path to \$B\$: \$A \to C \to B\$, Distance = **3**
- Shortest path to \$C\$: \$A \to C\$, Distance = **2**
- Shortest path to \$D\$: \$A \to C \to B \to D\$, Distance = **8**
- Shortest path to \$E\$: \$A \to C \to B \to D \to E\$, Distance = **10**
- Shortest path to \$Z\$: \$A \to C \to B \to D \to E \to Z\$, Distance = **13**

Key Takeaway Checklist for Exam Day

- [] For Infix to Postfix conversions, always show the complete step-by-step trace table.
- [] In AVL rotations, clearly mention whether it is LL, RR, LR, or RL rotation.
- [] Memorize the master sorting complexity table:
 - QuickSort: Avg $O(N \log N)$, Worst $O(N^2)$, Space $O(\log N)$
 - MergeSort: Avg $O(N \log N)$, Worst $O(N \log N)$, Space $O(N)$
 - HeapSort: Avg $O(N \log N)$, Worst $O(N \log N)$, Space $O(1)$